



BLOC 2 — RNCP 39583 · EXPERT EN DÉVELOPPEMENT LOGICIEL

Concevoir et développer des applications logicielles

Mise en œuvre des environnements, intégration et déploiement continu, prototype sécurisé, harnais de tests, recette et documentation technique — appliqués au projet fil rouge MotoTrack Pro.

Next.js 14 · App Router

TypeScript 5

PostgreSQL 15 · Prisma 5

Jest · Playwright

GitHub Actions · Vercel

RGAA · OWASP Top 10

Auteur

Établissement

Commanditaire

Version applicative

Date

Luca Straputicari

Ynov Campus — Master 2

MotoClub Alpes

1.0.0

21 avril 2026

A2.1	C2.1.1 — Environnements de déploiement et de test	Déploiement continu · KPI qualité/perf
A2.1	C2.1.2 — Système d'intégration continue	Fusion & test réguliers
A2.2	C2.2.1 — Prototype & architecture logicielle	Éliminatoire
A2.2	C2.2.2 — Harnais de tests unitaires	Éliminatoire
A2.2	C2.2.3 — Développement sécurisé & accessible	Éliminatoire · OWASP · RGAA
A2.2	C2.2.4 — Déploiement progressif & versionnage	Historique des versions
A2.3	C2.3.1 — Cahier de recettes	Éliminatoire
A2.3	C2.3.2 — Plan de correction des bogues	Analyse & traitement
A2.4	C2.4.1 — Documentation technique	Manuels déploiement / utilisation / MàJ
A	Annexe A — Glossaire technique	Définitions des termes
B	Annexe B — Table des figures	Index des captures insérées
C	Annexe C — Références & normes	Sources externes

► Contexte du projet

MotoTrack Pro est une application web de gestion de garage moto (suivi d'entretien, tableau de bord d'échéances, actualités, météo) développée pour le **MotoClub Alpes**. Elle cible en priorité le **web responsive** et le **mobile** via une Progressive Web App installable. Ce rapport couvre le Bloc 2 : de la mise en place des environnements outillés jusqu'à la documentation technique, en passant par le développement sécurisé et accessible, les tests et la recette.

142

Tests unitaires

22

Tests e2e

96%

Couverture (couche testée)

10/10Catégories OWASP
couvertes

Couche	Technologie	Rôle
Framework	Next.js 14 (App Router)	SSR, routage, API Routes
Langage	TypeScript 5 (strict)	Typage statique
Base de données	PostgreSQL 15 + Prisma 5	Persistence, migrations
Authentification	JWT (jose) + cookies httpOnly	Session sécurisée
Validation	Zod + React Hook Form	Contrôle des entrées
Tests	Jest + Testing Library · Playwright	Unitaire · e2e

Mettre en œuvre des environnements outillés (suivi de performance et de qualité) pour permettre le bon déroulement de la phase de développement.

► Environnement de développement

Composant	Outil	Rôle
Éditeur de code	Visual Studio Code	Développement, extensions ESLint/Prisma
Compilateur / transpileur	TypeScript (tsc) + SWC (via Next.js)	Typage statique et transpilation
Runtime	Node.js 20	Exécution serveur et outillage
Serveur d'application	Next.js (dev & prod), Vercel en production	SSR + API Routes
Base de données locale	PostgreSQL 15 (Docker) + Adminer	Persistance isolée et reproductible
Gestion de sources	Git + GitHub	Versionnage, revue, CI/CD
Gestion de paquets	npm (installation reproductible <code>npm ci</code>)	Dépendances verrouillées

📸 CAPTURE 1.1

VS Code ouvert sur le projet MotoTrack : arborescence `src/` visible à gauche + un fichier ouvert (ex. `src/lib/auth.ts`). Illustre l'éditeur de code.

📸 CAPTURE 1.2

Terminal `docker compose up -d` montrant les conteneurs `mototracker_db` (PostgreSQL) et `mototracker_adminer` démarrés.

► Outils de suivi de qualité et de performance

- **ESLint** (`next/core-web-vitals`) — qualité et bonnes pratiques React/Next, intégré à la CI.
- **Jest + couverture de code** — mesure quantitative de la qualité des tests (seuils bloquants dans `jest.config.js`).
- **Playwright** — tests end-to-end multi-navigateur avec rapport HTML.
- **Winston** — journalisation structurée (niveaux, JSON, horodatage).
- **Endpoint** `GET /api/health` — sonde de performance exposant l'état applicatif, l'état de la base et la **latence** (ms).

► Critères de qualité et de performance

Critère	Exigence projet	Outil / mesure
Couverture de tests (couche logique)	≥ 90 % statements	Jest — 95 %
Absence de régression	Suites 100 % vertes	Jest + Playwright en CI
Qualité du code	0 erreur de lint	ESLint
Intégrité du typage	0 erreur TypeScript	<code>tsc --noEmit</code>
Disponibilité / latence DB	latence < 100 ms	<code>/api/health</code>

 CAPTURE 1.3

Réponse de **GET /api/health** dans le navigateur ou via **curl** : `{ "status":"ok", "db":"connected", "latencyMs":<100 }`.

Configurer l'intégration continue : fusionner les codes sources et tester régulièrement les blocs de code pour réduire les risques de régression.

► Protocole d'intégration & de déploiement continus

Le pipeline `.github/workflows/ci.yml` s'exécute sur chaque `push` et chaque `pull request` vers `main` et `develop`. Il enchaîne des **séquences ordonnées et conditionnelles** :

```
push / PR → Job « test-and-build » (Ubuntu + service PostgreSQL 15)
  1. checkout du code
  2. npm ci (installation reproductible)
  3. prisma generate + migrate (préparation base éphémère)
  4. next lint (qualité)
  5. npm audit --audit-level=high (veille vulnérabilités – OWASP A06)
  6. jest (tests unitaires + couverture)
    └─ upload du rapport de couverture (artefact)
  7. next build (compilation production)
  8. playwright test (tests end-to-end)
    └─ upload du rapport Playwright (artefact)
      |
      ▼ si branche = main ET event = push ET job précédent vert
Job « deploy » → vercel --prod (déploiement continu)
```

RÉDUCTION DES RÉGRESSIONS

Aucun code n'atteint la production sans avoir franchi lint + 99 tests unitaires + build + 22 tests e2e. La fusion d'une PR est conditionnée à la réussite de ces séquences.

► Séquences d'intégration

Séquence	Déclencheur	Portée
Intégration	PR vers <code>develop</code> / <code>main</code>	lint, tests, build (validation)
Fusion	PR approuvée + CI verte	merge dans la branche cible
Déploiement	push sur <code>main</code>	mise en production Vercel

📸 CAPTURE 2.1

Onglet « Actions » de GitHub montrant un run du pipeline **vert** (toutes les étapes ✅). C'est la preuve visuelle des séquences d'intégration.

📸 CAPTURE 2.2

Détail d'un job (test-and-build) avec les étapes lint / jest / build / playwright dépliées, ou une **Pull Request** affichant les checks obligatoires réussis.

Concevoir un prototype fonctionnel, structuré et maintenable, tenant compte de l'ergonomie, des équipements ciblés et des exigences de sécurité.

► Architecture & maintenabilité

L'application suit une architecture **en couches** favorisant la maintenabilité et la séparation des responsabilités :

Couche	Emplacement	Responsabilité
Présentation	<code>src/app/**/page.tsx</code> , <code>src/components</code>	Pages et composants UI réutilisables
API / métier	<code>src/app/api/**/route.ts</code>	Endpoints REST, règles métier
Sécurité transverse	<code>src/middleware.ts</code> , <code>src/lib/auth.ts</code>	Protection des routes, JWT
Accès aux données	<code>src/lib/prisma.ts</code> , <code>prisma/schema.prisma</code>	ORM, modèle de données
Utilitaires	<code>src/lib/*</code>	Formatage, logger, helpers

► Frameworks & paradigmes

- **Next.js 14 App Router** — rendu serveur, routage par convention de fichiers.
- **Programmation par composants** React (composition, `forwardRef`).
- **Validation déclarative** avec Zod (schémas typés réutilisés côté client et serveur).
- **Design system** (Tailwind + variantes `cva`) pour la cohérence UI.

Cette organisation applique le principe de **séparation des préoccupations** : une modification de l'interface n'impacte pas les règles métier, et une évolution de la base de données reste confinée à la couche d'accès aux données via Prisma. Le **typage statique strict** de TypeScript, activé sur l'ensemble du projet (`tsc --noEmit` sans erreur), constitue un premier filet de sécurité pour la maintenabilité : toute rupture de contrat entre couches est détectée à la compilation. Les schémas Zod sont **partagés** entre le client (React Hook Form) et le serveur (routes API), ce qui évite la duplication des règles de validation et garantit leur cohérence — un gage de maintenabilité important.

► Fonctionnalités principales & user stories

Le prototype couvre un ensemble cohérent de fonctionnalités traduisant directement les user stories exprimées par le commanditaire. Chacune est fonctionnelle et manipulable de bout en bout :

User story	Fonctionnalité réalisée	Composants d'interface
« En tant que motard, je veux créer un compte sécurisé »	Inscription / connexion / déconnexion (JWT)	Formulaires, champs, boutons, messages d'erreur
« ... gérer plusieurs motos »	Garage multi-motos, wizard 3 étapes, moto principale	Assistant multi-étapes, cartes, badges, menus
« ... suivre l'entretien et ne rater aucune échéance »	Carnet d'entretien, MàJ auto du kilométrage, échéances calculées	Formulaire, listes filtrables, tableau de bord
« ... trouver des pièces et suivre l'actualité »	Modules Pièces, Actualités (RSS), Météo	Liens dynamiques, cartes d'articles, vignettes

L'ergonomie tient compte des **équipements ciblés** : interface responsive web et **application installable (PWA)** sur mobile, navigation adaptée (menu latéral sur desktop, barre défilante sur mobile). Les **exigences**

de sécurité sont intégrées dès la conception (défense en profondeur détaillée en C2.2.3).

 **CAPTURE 3.1 À 3.4**

4 captures de l'application en fonctionnement : (a) accueil (scène 3D), (b) tableau de bord, (c) **Mon Garage** avec le wizard, (d) carnet d'entretien. Preuve du prototype fonctionnel et de ses composants d'interface.

 **CAPTURE 3.5**

Arborescence du projet `src/` annotée (couches `app / api / lib / middleware / components`) illustrant l'architecture en couches.

► **Auto-évaluation critère par critère**

Critère du référentiel	Preuve	Statut
Bonnes pratiques (framework, paradigmes)	Next.js 14, React par composants, validation déclarative Zod, design system <code>cva</code>	Atteint
Prototype fonctionnel répondant aux besoins	7 modules opérationnels, recette 15/15 (C2.3.1)	Atteint
Ensemble cohérent de fonctionnalités + user stories	Mapping user stories ci-dessus	Atteint
Composants d'interface présents et fonctionnels	Formulaires, boutons, menus, wizard, cartes	Atteint
Exigences de sécurité satisfaites	OWASP Top 10 couvert (C2.2.3)	Atteint

REGARD CRITIQUE

L'architecture en couches est saine. La dette technique initialement identifiée — des pages concentrant présentation ET logique (ex. le wizard `garage/new`) — a été **traitée** : la logique métier (schéma, validations, filtrage, mapping d'étapes, parsing d'erreurs) a été extraite dans un module pur `wizardLogic.ts`, désormais **couvert par des tests unitaires**. Il reste à poursuivre cette démarche sur les autres écrans denses (tableau de bord, météo) pour homogénéiser la testabilité — axe d'amélioration continue, sans impact sur le caractère fonctionnel du prototype.

Développer un harnais de tests unitaires pour prévenir les régressions et garantir le bon fonctionnement du logiciel.

► Stratégie de test & interprétation du critère

Le critère éliminatoire exige que « *les tests unitaires couvrent la majorité du code développé* ». Ce critère mérite une lecture rigoureuse : dans une application Next.js, tout le code n'est pas de même nature. On distingue une **couche logique** (bibliothèques, routes API, middleware, validation, composants) — testable en isolation par des tests unitaires — et une **couche de présentation** lourde (pages intégrant appels réseau, rendu 3D, état complexe) qui relève par nature du test **end-to-end**. La stratégie retenue est donc explicitement à deux niveaux :

- **Tests unitaires (Jest)** — 142 tests ciblant la couche logique et les parcours critiques (authentification, navigation), là où se concentrent les règles métier et la sécurité.
- **Tests end-to-end (Playwright)** — 22 tests validant les pages et les parcours utilisateur réels (voir C2.3.1).

Le périmètre de mesure unitaire (`collectCoverageFrom` dans `jest.config.js`) est **documenté et assumé**, avec des **seuils bloquants** en CI (statements 90 %, branches 80 %, fonctions 90 %, lignes 90 %). Cette configuration est ouverte à la vérification du jury.

142

Tests unitaires

96%

Statements (couche testée)

96%

Lignes

87%

Branches

► Transparence sur la couverture globale

Par honnêteté intellectuelle, deux chiffres sont présentés. Sur le **périmètre ciblé** (logique + parcours critiques), la couverture atteint **96 % des instructions**. Sur l'**intégralité** du dossier `src/` (pages lourdes comprises), elle s'établit désormais à **≈ 55 % des instructions (55 % des lignes)** — soit une **majorité du code développé**, ce qui satisfait le critère au sens strict. Ce résultat a été obtenu en refactorisant la logique des pages complexes (ex. le wizard `garage/new`) dans des modules purs et testables. Le reliquat non couvert unitairement correspond au JSX de présentation, validé par les tests e2e : la combinaison unitaire + e2e couvre donc l'essentiel du code.

► Périmètre couvert

Module testé	Cas couverts
<code>api/auth/login · register · logout</code>	succès, 401, 400 (Zod), 409, 500, cookie httpOnly, hachage bcrypt
<code>api/motorcycles · [id]</code>	401/400/404, ownership, moto principale
<code>api/maintenances</code>	401/403/400/500, filtre, MàJ auto kilométrage
<code>api/health</code>	200 (db ok) / 503 (db injoignable)
<code>api/news · motorcycle-models</code>	agrégation RSS, dédoublonnage, appel NHTSA, résilience
<code>middleware</code>	redirections, jeton valide/invalid/absent
<code>lib</code> (auth, formatPlate, utils, logger)	JWT, format SIV, fusion de classes, logs
Composants UI (button, input, label)	variantes, props, <code>asChild</code>

PRÉVENTION DES RÉGRESSIONS

Chaque anomalie corrigée est accompagnée d'un test dédié (approche non-régression). Le harnais s'exécute automatiquement à chaque push (CI), bloquant tout code qui casserait un comportement existant.

► Auto-évaluation critère par critère

Critère du référentiel	Preuve	Statut
Jeu de tests couvrant une fonctionnalité demandée	Authentification testée de bout en bout (route + page + middleware)	Atteint
Les tests couvrent la majorité du code développé	96 % de la couche logique et 55 % du code global (majorité) + complément e2e	Atteint

REGARD CRITIQUE

Le point initialement le plus exposé — une couverture globale minoritaire ($\approx 44\%$) — a été **corrigé** : le refactoring de la logique des pages complexes dans des modules purs a porté la couverture globale à **55 %** (majorité atteinte au sens strict), sans dénaturer la stratégie de test. La marge restante concerne le JSX de présentation pur, couvert par les tests e2e ; sa montée en couverture unitaire relèverait de tests de rendu supplémentaires, à l'utilité marginale au regard du risque.

CAPTURE 4.1

Sortie de `npm run test` : tableau de couverture Jest (All files $\approx 95\%$) + **Tests: 142 passed** , **Test Suites: 23 passed** .

CAPTURE 4.2

Rapport HTML de couverture : ouvrir `coverage/lcov-report/index.html` et capturer la vue d'ensemble (barres de couverture par dossier).

Développer en veillant à la sécurisation du code (OWASP Top 10) et aux exigences d'accessibilité (RGAA).

► **Sécurité — couverture des 10 failles OWASP (2021)**

Risque OWASP	Mesure mise en œuvre
A01 Contrôle d'accès	Middleware + filtrage par <code>userId</code> + contrôle d'ownership (404/403)
A02 Cryptographie	bcrypt (coût 10), JWT HS256, cookie <code>httpOnly</code> / <code>secure</code>
A03 Injection	Prisma (requêtes paramétrées) + validation Zod + échappement React
A04 Conception	Défense en profondeur (validation + accès + ownership)
A05 Configuration	Secrets externalisés + en-têtes HTTP (X-Frame-Options, HSTS, CSP Report-Only) + erreurs génériques
A06 Composants	Dépendances épinglées + <code>npm audit</code> en CI
A07 Authentification	bcrypt.compare, expiration de session + rate limiting (429) sur l'auth
A08 Intégrité	<code>package-lock</code> + pipeline CI contrôlé + JWT signé
A09 Journalisation	Winston structuré + <code>/api/health</code>
A10 SSRF	URL sortantes constantes et encodées (NHTSA, RSS)

Détail complet et pistes d'amélioration : [docs/securite-owasp.md](#) . En-têtes, audit CI et rate limiting sont désormais **implémentés et testés**.

► **Accessibilité — référentiel RGAA**

Le **RGAA** (Référentiel Général d'Amélioration de l'Accessibilité) a été retenu car il constitue la norme **officielle française**, alignée sur les WCAG 2.1, adaptée à un service à destination d'une association. Actions mises en œuvre :

Critère RGAA	Action
8.3 — Langue	<code><html lang="fr"></code>
9.1 — Structure de titres	Un <code><h1></code> unique par page
11.1 / 11.13 — Formulaires	Étiquettes <code><Label htmlFor></code> + <code>autocomplete</code>
11.10 — Erreurs de saisie	<code>role="alert"</code> , <code>aria-invalid</code> , <code>aria-describedby</code>
12.7 — Lien d'évitement	« Aller au contenu principal » vers <code>#main-content</code>
1.2 — Images décoratives	Icônes <code>aria-hidden</code>
12.2 — Navigation	<code><nav aria-label></code> + <code>aria-current="page"</code>

Le choix du RGAA plutôt qu'OPQUAST est **justifié** : OPQUAST est une check-list de qualité web généraliste, tandis que le RGAA est spécifiquement dédié à l'accessibilité numérique, opposable et aligné sur les WCAG 2.1 — plus pertinent pour démontrer la prise en compte des personnes en situation de handicap. Les correctifs ont été appliqués sur les **parcours critiques** (authentification, navigation principale) et sont

prouvés par des tests automatisés. Une **grille d'audit chiffrée** (`docs/accessibilite-rgaa.md`) évalue 18 critères RGAA sur ce périmètre : **100 % de conformité** sur les 17 critères applicables.

ACCESSIBILITÉ TESTÉE AUTOMATIQUEMENT

5 tests e2e dédiés (`tests/e2e/accessibility.spec.ts`) vérifient langue, étiquettes, autocomplétion, signalement d'erreurs (role=alert / aria-invalid) et structure de titres — la conformité est donc **vérifiable**, non déclarative.

► **Auto-évaluation critère par critère**

Critère du référentiel	Preuve	Statut
Couvrir les 10 failles principales de l'OWASP	Tableau A01→A10 ci-dessus + <code>securite-owasp.md</code>	Atteint
Référentiel d'accessibilité présenté et justifié	RGAA choisi et argumenté vs OPQUAST	Atteint
Le prototype répond aux exigences du référentiel	Grille RGAA 100 % (17/17 critères applicables) + 5 tests e2e verts	Atteint sur les parcours critiques

REGARD CRITIQUE

Côté **OWASP**, le socle couvre désormais les dix catégories avec des mesures effectives et testées ; les axes restants (MFA, CSP en mode bloquant, RBAC administrateur) relèvent d'une sécurité avancée non exigée à ce stade. Côté **RGAA**, la conformité est chiffrée (100 %) mais sur un **périmètre volontairement restreint** aux parcours critiques : elle ne constitue pas encore un audit exhaustif des 106 critères sur toutes les pages (météo, pièces), ni une déclaration de conformité formelle. C'est la limite honnête du présent niveau ; elle répond néanmoins à l'exigence « le prototype répond aux exigences du référentiel établi » sur le périmètre défini et documenté.

 **CAPTURE 5.1**

Code de sécurité : `register/route.ts` (bcrypt) et le bloc cookie `httpOnly/secure` de `login/route.ts`.

 **CAPTURE 5.2**

Audit Lighthouse — onglet Accessibilité (DevTools Chrome) + tests `accessibility.spec.ts` verts.

Déployer le logiciel à chaque modification, de façon progressive, en garantissant une solution stable et manipulable en autonomie.

► **Gestion de versions**

- **Git + GitHub** : historique complet, branches `main` / `develop` / `feature` .
- **Semantic Versioning** + `CHANGELOG.md` (0.1.0 → 1.0.0) traçant chaque évolution du prototype.
- **Tags Git** matérialisant chaque release.

► **Déploiement progressif**

Chaque `push` sur `main` déclenche, après validation automatique, un déploiement Vercel. Les déploiements sont **immuables et réversibles** (rollback en un clic vers une version antérieure stable).

Version	Date	Apport principal
0.1.0	15/03/2026	Init Next.js, auth, garage, entretien
0.2.0	24/03/2026	Pièces, scène 3D, design dark
0.3.0	07/04/2026	Wizard moto, NHTSA, actualités
1.0.0	21/04/2026	Version stable, CI/CD, supervision, tests

UTILISABLE EN AUTONOMIE

L'application est fonctionnelle et manipulable sans assistance (cf. manuel d'utilisation) : parcours d'inscription, garage, entretien et tableau de bord validés en recette.

 CAPTURE 6.1

Graphe de commits GitHub (onglet « Commits » ou « Insights ») et/ou la page **Releases** avec les tags de version.

 CAPTURE 6.2

Historique des déploiements Vercel (onglet Deployments) montrant les mises en production successives.

Élaborer le cahier de recettes (scénarios de test et résultats attendus) pour détecter anomalies et régressions.

► Structure & couverture

Le cahier de recettes ([docs/cahier-recettes.md](#)) formalise **17 scénarios (RT-01 à RT-17)** couvrant l'ensemble des fonctionnalités attendues, organisés par module. Chaque scénario précise préconditions, étapes, résultat attendu, résultat obtenu et statut, et est **relié à son test automatisé** (table de traçabilité en fin de cahier).

Module	Scénarios	Type de test
Authentification	RT-01 → RT-05	Fonctionnel + sécurité (accès protégé)
Gestion du garage	RT-06 → RT-09	Fonctionnel + structurel (validation SIV)
Entretien	RT-10 → RT-12	Fonctionnel (MàJ kilométrage, filtre)
Tableau de bord	RT-13 → RT-14	Fonctionnel (calculs d'échéances)
Supervision	RT-15	Technique (endpoint santé)
Sécurité	RT-16 → RT-17	Sécurité (isolation des données, jeton invalide)

► Automatisation de la recette (Playwright)

Les scénarios sont **rejoués automatiquement** par 22 tests e2e Playwright, garantissant la reproductibilité et la détection des régressions :

- [auth.spec.ts](#) — connexion, inscription, validation, protection des routes (RT-01→05).
- [home.spec.ts](#) — page d'accueil et navigation.
- [accessibility.spec.ts](#) — tests de sécurité d'accès et d'accessibilité.
- [api-health.spec.ts](#) — contrat de l'endpoint de supervision (RT-15).

► Typologie des tests exécutés

Le critère impose la présence de tests **fonctionnels, structurels et de sécurité** conformes au plan. Le cahier et son automatisation couvrent explicitement ces trois natures :

Type de test	Objet	Scénarios / preuves
Fonctionnel	Conformité au comportement attendu	RT-01, 03, 06, 08, 10 à 14 (parcours métier)
Structurel	Validation des données & règles de saisie	RT-07 (format SIV), RT-02 (unicité email), schémas Zod
Sécurité	Contrôle d'accès & authentification	RT-04 (identifiants erronés), RT-05 (route protégée), redirections middleware
Technique	Supervision / disponibilité	RT-15 (endpoint santé)

BILAN DE RECETTE

17/17 scénarios passés, 0 anomalie fonctionnelle (validé par le MotoClub Alpes le 18/04/2026), rejoués automatiquement par 22 tests Playwright et le harnais unitaire pour prévenir les régressions.

► **Auto-évaluation critère par critère**

Critère du référentiel	Preuve	Statut
Le cahier reprend l'ensemble des fonctionnalités attendues	17 scénarios sur les 6 modules du périmètre	Atteint
Tests fonctionnels, structurels et de sécurité conformes au plan	Typologie ci-dessus + module Sécurité (RT-16/17) + traçabilité vers les tests	Atteint

REGARD CRITIQUE

La faiblesse initialement identifiée — un cahier trop **fonctionnel** — a été corrigée par l'ajout d'un module **Sécurité** explicite (RT-16 isolation des données, RT-17 jeton invalide) et d'une **table de traçabilité** reliant chaque scénario à son test automatisé. La marge de progression restante concerne des jeux de données limites plus systématiques (fuzzing léger des formulaires) et une seconde campagne de recette datée après corrections, pour matérialiser le cycle complet détection → correction → re-test.

CAPTURE 7.1

Extrait du cahier de recettes ([docs/cahier-recettes.md](#)) : tableau d'un module avec les statuts .

CAPTURE 7.2

Rapport Playwright : sortie de `npm run test:e2e` (22 passed) et/ou le rapport HTML ([playwright-report](#)).

Élaborer un plan de correction à partir de l'analyse des anomalies et régressions détectées, pour garantir un fonctionnement conforme.

► Processus de traitement

Chaque anomalie suit un cycle : **détection** → **qualification (sévérité)** → **analyse de cause racine** → **correction avec test de non-régression** → **vérification** → **clôture** (tracée dans `CHANGELOG.md`). Détail : [docs/plan-correction-bogues.md](#).

► Registre des anomalies traitées

ID	Anomalie	Sévérité	Statut
BUG-01	Couverture de tests insuffisante (6 %)	Bloquant	✓ 95 %
BUG-02	Jest exécutait les tests Playwright	Majeur	✓ Résolu
BUG-03	Client Prisma périmé (typage <code>isPrimary</code>)	Bloquant	✓ Résolu
BUG-04	Lint CI bloquant (config absente)	Majeur	✓ Résolu
BUG-05	Formulaires non accessibles	Majeur	✓ Résolu
BUG-06	Incohérence de nommage de l'app	Mineur	✓ Résolu

ANALYSE DES POINTS D'AMÉLIORATION

Chaque test en échec a fait l'objet d'une analyse de cause racine et d'un correctif accompagné d'un test automatisé de non-régression, conformément à l'attendu.

📸 CAPTURE 8.1

GitHub Issues : une *issue* qualifiée (labels sévérité) et son correctif lié (commit / PR de fermeture). Illustre la détection → qualification → traitement.

Rédiger la documentation technique d'exploitation pour assurer la traçabilité et le suivi des évolutions.

► **Corpus documentaire livré**

Document	Fichier	Contenu
Manuel de déploiement	<code>docs/manuel-deploiement.md</code>	Architecture cible, prérequis, variables, déploiement local & Vercel, rollback
Manuel d'utilisation	<code>docs/manuel-utilisation.md</code>	Parcours utilisateur pas à pas, PWA, accessibilité
Manuel de mise à jour	<code>docs/manuel-mise-a-jour.md</code>	Versionnage, Git Flow, migrations Prisma, checklist de publication
Sécurité OWASP	<code>docs/securite-owasp.md</code>	Cartographie des 10 risques et mesures
Grille d'audit RGAA	<code>docs/accessibilite-rgaa.md</code>	18 critères évalués, taux de conformité
Plan de correction	<code>docs/plan-correction-bogues.md</code>	Processus et registre des anomalies
README & CHANGELOG	<code>README.md</code> , <code>CHANGELOG.md</code>	Prise en main, choix technologiques et de langages, historique des versions

TRAÇABILITÉ DES CHOIX

La documentation justifie les choix de technologies (Next.js, TypeScript, PostgreSQL/Prisma, JWT, Zod) et de langages, et décrit les procédures d'exploitation pour les futures évolutions.

► **Synthèse de conformité — Bloc 2**

Compétence	Livrable principal	Statut
C2.1.1	Environnements + CI/CD + KPI	✓
C2.1.2	Pipeline d'intégration continue	✓
C2.2.1 (élim.)	Prototype + architecture	✓
C2.2.2 (élim.)	Harnais de tests (95 %)	✓
C2.2.3 (élim.)	OWASP + RGAA	✓
C2.2.4	Versionnage + déploiement	✓
C2.3.1 (élim.)	Cahier de recettes	✓
C2.3.2	Plan de correction des bogues	✓
C2.4.1	Manuels & documentation	✓

 CAPTURE 9.1

Aperçu des manuels : capture d'un des manuels rendus (déploiement ou utilisation) montrant la clarté rédactionnelle et la structure.

Définition des principaux termes techniques employés dans ce rapport.

Terme	Définition
API Route	Point d'entrée serveur de Next.js (fichier <code>route.ts</code>) exposant une fonction HTTP (GET, POST...); contient les règles métier côté serveur.
App Router	Système de routage de Next.js 14 fondé sur l'arborescence de fichiers du dossier <code>app/</code> , avec rendu serveur par défaut.
bcrypt	Algorithme de hachage de mots de passe avec « sel » et facteur de coût, résistant aux attaques par force brute.
CI/CD	Intégration Continue / Déploiement Continu : automatisation de la vérification (tests, build) et de la mise en production à chaque modification du code.
Cookie httpOnly	Cookie inaccessible au JavaScript du navigateur, protégeant le jeton de session contre le vol par script (XSS).
Couverture (coverage)	Pourcentage du code exécuté par les tests. Mesurée en <i>statements</i> (instructions), <i>branches</i> , <i>functions</i> et <i>lines</i> .
cva	<i>class-variance-authority</i> : utilitaire de gestion des variantes de style d'un composant (taille, couleur...).
e2e (end-to-end)	Test de bout en bout simulant un utilisateur réel dans un navigateur, du clic au résultat affiché.
ESLint	Analyseur statique de code JavaScript/TypeScript appliquant des règles de qualité et de bonnes pratiques.
Jest	Framework de tests unitaires JavaScript, ici couplé à Testing Library pour le rendu de composants React.
JWT	<i>JSON Web Token</i> : jeton signé encodant l'identité de l'utilisateur, vérifié à chaque requête pour authentifier la session.
jose	Librairie de création et vérification de jetons JWT compatible avec l'environnement Edge de Next.js.
Lighthouse	Outil d'audit intégré à Chrome mesurant performance, accessibilité, bonnes pratiques et SEO d'une page.
Middleware	Fonction exécutée avant l'accès à une route ; ici, filtre d'authentification protégeant les pages privées.
Migration	Script versionné décrivant une évolution du schéma de base de données, rejouable de façon reproductible (Prisma).
Mock	Objet de substitution simulant une dépendance (base de données, API externe) pour isoler le code testé.
NHTSA	<i>National Highway Traffic Safety Administration</i> : API publique fournissant la liste officielle des modèles de véhicules.
ORM	<i>Object-Relational Mapping</i> : couche traduisant les objets du code en requêtes SQL (ici Prisma), avec requêtes paramétrées.
OWASP	<i>Open Worldwide Application Security Project</i> : organisation de référence en sécurité applicative, éditrice du « Top 10 » des risques.

Playwright	Framework de tests end-to-end multi-navigateur, avec rapport HTML et exécution en CI.
Prisma	ORM TypeScript pour PostgreSQL : schéma déclaratif, migrations, client typé.
PWA	<i>Progressive Web App</i> : application web installable sur mobile, utilisable comme une application native.
RGAA	Référentiel Général d'Amélioration de l'Accessibilité : norme française d'accessibilité numérique, alignée sur les WCAG.
Rollback	Retour à une version antérieure stable de l'application en cas d'anomalie en production.
SIV	Système d'Immatriculation des Véhicules : format français des plaques (AB-123-CD).
SSR	<i>Server-Side Rendering</i> : génération du HTML côté serveur avant envoi au navigateur (meilleur SEO et temps d'affichage).
Tailwind CSS	Framework CSS « utilitaire » : la mise en forme s'exprime via des classes prédéfinies dans le balisage.
Testing Library	Bibliothèque de test de composants React centrée sur l'usage réel (rôles, libellés) plutôt que sur l'implémentation.
TypeScript	Sur-ensemble typé de JavaScript ; le typage statique détecte des erreurs dès la compilation.
Vercel	Plateforme d'hébergement et de déploiement continu optimisée pour Next.js.
Winston	Bibliothèque de journalisation structurée (niveaux, horodatage, format JSON).
Zod	Bibliothèque de validation de schémas TypeScript ; garantit la conformité des données entrantes.

Index des captures d'écran et illustrations à insérer dans le rapport, avec leur emplacement (compétence associée).

Fig. 1.1	VS Code ouvert sur le projet (environnement de développement)	C2.1.1
Fig. 1.2	Conteneurs Docker PostgreSQL + Adminer démarrés	C2.1.1
Fig. 1.3	Réponse de l'endpoint <code>GET /api/health</code>	C2.1.1
Fig. 2.1	Pipeline GitHub Actions vert (onglet Actions)	C2.1.2
Fig. 2.2	Détail d'un job CI / checks d'une Pull Request	C2.1.2
Fig. 3.1	Page d'accueil avec scène 3D	C2.2.1
Fig. 3.2	Tableau de bord (échéances, dépenses)	C2.2.1
Fig. 3.3	Mon Garage — wizard d'ajout de moto	C2.2.1
Fig. 3.4	Carnet d'entretien	C2.2.1
Fig. 3.5	Arborescence du projet <code>src/</code> annotée	C2.2.1
Fig. 4.1	Sortie <code>npm run test</code> (couverture + 142 tests)	C2.2.2
Fig. 4.2	Rapport HTML de couverture (<code>lcov-report</code>)	C2.2.2
Fig. 5.1	Code de sécurité (bcrypt, cookie httpOnly)	C2.2.3
Fig. 5.2	Audit Lighthouse — accessibilité + tests a11y verts	C2.2.3
Fig. 6.1	Graphe de commits / page Releases GitHub	C2.2.4
Fig. 6.2	Historique des déploiements Vercel	C2.2.4
Fig. 7.1	Extrait du cahier de recettes	C2.3.1
Fig. 7.2	Rapport Playwright (22 tests e2e)	C2.3.1
Fig. 8.1	GitHub Issue qualifiée + correctif lié	C2.3.2
Fig. 9.1	Aperçu d'un manuel technique rendu	C2.4.1

CONVENTION

Chaque emplacement de capture est matérialisé dans le corps du rapport par un cadre en pointillés bleus. Remplacer le cadre par l'image correspondante en conservant sa légende.

Sources externes, référentiels et documentations mobilisés.

▸ Sécurité & accessibilité

- **OWASP Top 10 (2021)** — owasp.org/Top10/
- **RGAA** (Référentiel Général d'Amélioration de l'Accessibilité) — accessibilite.numerique.gouv.fr
- **WCAG 2.1** (Web Content Accessibility Guidelines) — w3.org/TR/WCAG21/

▸ Documentations techniques

- **Next.js 14** — nextjs.org/docs
- **Prisma ORM** — prisma.io/docs
- **Jest** — jestjs.io/docs · **Playwright** — playwright.dev
- **Zod** — zod.dev
- **API NHTSA vPIC** — vpic.nhtsa.dot.gov/api

▸ Méthodologie & conventions

- **Semantic Versioning 2.0.0** — semver.org/lang/fr/
- **Référentiel RNCP 39583** — Expert en développement logiciel, Bloc 2.

FIN DU DOCUMENT

Rapport Bloc 2 — MotoTrack Pro · Luca Straputicari · Ynov Campus · 21 avril 2026.